

Compte Rendu
TP N° 3
**Gestion des Utilisateurs
& Authentification Web**

J2EE – Java EE (JEE) / Jakarta EE

Étudiante Haiti Aya

Module J2EE / Jakarta EE

Filière TDIA2 – Semestre 4

Année 2025 – 2026

Encadrant Pr. Mohamed Cherradi

Java Servlets

JSP

Sessions HTTP

Filters

Apache Tomcat

MVC

Table des matières

1	Introduction	2
1.1	Objectif général.....	2
1.2	Technologies employées.....	2
2	Architecture du Projet	3
2.1	Modèle MVC.....	3
2.2	Structure des composants.....	4
3	Fonctionnalités Développées	5
3.1	Système d'inscription (Sign Up)	5
3.1.1	Page JSP.....	5
3.1.2	Servlet associée.....	5
3.2	Système de connexion (Login).....	5
3.2.1	Page JSP.....	6
3.2.2	Servlet associée.....	6
3.3	Déconnexion (Logout)	6
3.4	Dashboard (espace sécurisé).....	7
3.5	Sécurité par filtre HTTP (AuthFilter).....	7
3.6	Gestion des utilisateurs (UserStore).....	8
4	Cycle de Fonctionnement	10
4.1	Diagramme de flux.....	11
4.2	Description étape par étape.....	11
5	Difficultés & Solutions	13
6	Résultats & Conclusion	14
6.1	Résultats obtenus.....	14
6.2	Conclusion	14

1. Introduction

Contexte du TP

Ce troisième travail pratique du module **J2EE / Jakarta EE** porte sur le développement d'une application web complète de gestion des utilisateurs. L'application met en œuvre un système d'authentification robuste couvrant l'inscription, la connexion, la déconnexion et la protection des ressources sensibles par filtrage HTTP.

1.1 Objectif général

L'objectif principal de ce TP est de concevoir et d'implémenter une application web sécurisée en Java permettant :

- L'**inscription** de nouveaux utilisateurs (*Sign Up*)
- La **connexion** à un espace personnel (*Login*)
- La **déconnexion** sécurisée (*Logout*)
- L'accès protégé à un **tableau de bord** (*Dashboard*)

1.2 Technologies employées

Technologie	Rôle dans le projet
Java Servlets	Contrôleurs HTTP gérant la logique métier
JSP	Génération dynamique des vues HTML
Sessions HTTP	Maintien de l'état utilisateur entre requêtes
Servlet Filters	Sécurisation des ressources protégées
Apache Tomcat	Serveur d'application Java EE
Eclipse EE	Environnement de développement

Table 1.1. Technologies utilisées dans le TP3

2. Architecture du Projet

2.1 Modèle MVC

Le projet adopte une architecture **Modèle-Vue-Contrôleur (MVC)** enrichie d'une couche de sécurité. Ce pattern sépare nettement les responsabilités et favorise la maintenabilité du code.

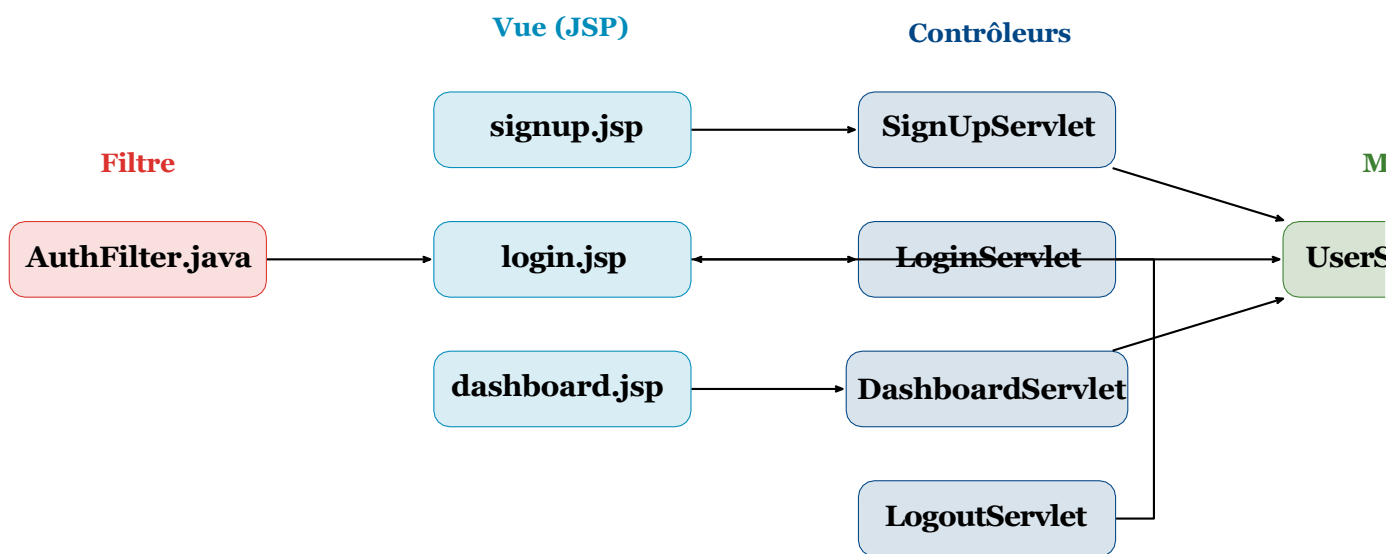


Figure 2.1. Architecture MVC du projet TP3

2.2 Structure des composants

Couche	Fichier	Responsabilité
Modèle	UserStore.java	Stockage en mémoire et vérification des utilisateurs
Vue	signup.jsp	Formulaire d'inscription
Vue	login.jsp	Formulaire de connexion
Vue	dashboard.jsp	Interface utilisateur connecté
Contrôleur	SignUpServlet.java	Traitement de l'inscription
Contrôleur	LoginServlet.java	Vérification et ouverture de session
Contrôleur	LogoutServlet.java	Invalidation de session
Contrôleur	DashboardServlet.java	Accès à l'espace protégé
Sécurité	AuthFilter.java	Filtrage des requêtes HTTP

Table 2.1. Répartition des composants selon l'architecture MVC

3. Fonctionnalités Développées

3.1 Système d'inscription (Sign Up)

3.1.1 Page JSP

La page signup.jsp présente un formulaire permettant à l'utilisateur de saisir ses informations personnelles (nom d'utilisateur, mot de passe, etc.) et de soumettre sa demande de création de compte.

3.1.2 Servlet associée

La SignUpServlet intercepte la requête POST, récupère les paramètres du formulaire et délègue le stockage à UserStore.

```
1  @WebServlet("/ signup ")
2  public class Sign UpServlet extends HttpServlet {
3
4      @Override
5      protected void doPost(HttpServletRequest request,
6                          HttpServletResponse response)
7          throws ServletException, IOException {
8
9          String username = request.getParameter(" username ");
10         String password = request.getParameter(" password ");
11
12         // Enregistrement de l'utilisateur en m moire
13         UserStore .add User( username , password );
14
15         // Redirection vers la page de connexion
16         response .send Redirect(" login .jsp ");
17     }
18 }
```

Listing 3.1. Traitement de l'inscription – SignUpServlet.java

3.2 Système de connexion (Login)

3.2.1 Page JSP

La page login.jsp propose un formulaire de connexion avec les champs *username* et *password*.

3.2.2 Servlet associée

LoginServlet vérifie les identifiants via UserStore, puis crée une session HTTP en cas de succès.

```
1 @WebServlet("/login")
2 public class LoginServlet extends HttpServlet {
3
4     @Override
5     protected void doPost(HttpServletRequest request,
6                           HttpServletResponse response)
7         throws ServletException, IOException {
8
9         String username = request.getParameter("username");
10        String password = request.getParameter("password");
11
12        if (UserStore.authenticate(username, password)) {
13            // Cr ation de la session utilisateur
14            request.getSession()
15                .setAttribute("user", username);
16            response.sendRedirect("dashboard");
17        } else {
18            response.sendRedirect("login.jsp?error=1");
19        }
20    }
21 }
```

Listing 3.2. Authentification et création de session – LoginServlet.java

3.3 Déconnexion (Logout)

LogoutServlet invalide la session active et redirige l'utilisateur vers la page de connexion.

```
1 @WebServlet("/logout")
2 public class LogoutServlet extends HttpServlet {
3
4     @Override
5     protected void doGet(HttpServletRequest request,
```

```
6         HttpServletResponse response )
7         throws ServletException , IOException {
8
9         // Destruction de la session
10        request.getSession ().invalidate ();
11        response.sendRedirect("login.jsp");
12    }
13 }
```

Listing 3.3. Invalidation de session – LogoutServlet.java

3.4 Dashboard (espace sécurisé)

Le tableau de bord affiche les informations de l'utilisateur connecté. Il n'est accessible qu'aux utilisateurs authentifiés — le filtre AuthFilter garantit cette restriction.

```
1 @WebServlet("/ dashboard ")
2 public class DashboardServlet extends HttpServlet {
3
4     @Override
5     protected void doGet( HttpServletRequest request,
6                           HttpServletResponse response )
7         throws ServletException , IOException {
8
9         String user = (String) request.getSession ()
10                        .getAttribute("user")
11                        ;
12        request.setAttribute("username", user);
13        request.getRequestDispatcher("dashboard.jsp")
14            .forward(request, response);
15    }
16 }
```

Listing 3.4. Affichage du dashboard – DashboardServlet.java

3.5 Sécurité par filtre HTTP (AuthFilter)

AuthFilter constitue la **couche de sécurité centrale** de l'application. Il intercepte toutes les requêtes et vérifie la présence d'une session valide avant de laisser passer la requête.

```
1 @WebFilter("/ dashboard /*")
2 public class AuthFilter implements Filter {
```



```

3
4  @Override
5  public void doFilter(ServletRequest req,
6                      ServletResponse res,
7                      FilterChain chain)
8                      throws IOException, ServletException {
9
10     HttpServletRequest request = (HttpServletRequest)
11         req;
12     HttpServletResponse response = (HttpServletResponse)
13         res;
14
15     HttpSession session = request.getSession(false);
16
17     if (session == null
18         || session.getAttribute("user") == null) {
19         // Utilisateur non authentifié redirection
20         response.sendRedirect(
21             request.getContextPath() + "/login.jsp");
22     } else {
23         // Utilisateur authentifié passage
24         chain.doFilter(req, res);
25     }
26 }

```

Listing 3.5. Filtre de sécurité – AuthFilter.java

3.6 Gestion des utilisateurs (UserStore)

UserStore est la classe modèle du projet. Elle maintient en mémoire la liste des utilisateurs inscrits et expose les méthodes nécessaires aux servlets.

```

1 public class UserStore {
2
3     // Stockage clé-valeur : username -> password
4     private static final Map<String, String> users
5         = new HashMap<>();
6
7     public static void addUser(String username,
8                               String password) {
9         users.put(username, password);
10    }
11 }

```

```
10     }  
11  
12     public static boolean authenticate(String username,  
13                                     String password) {  
14         return password.equals(users.get(username));  
15     }  
16 }
```

Listing 3.6. Stockage en mémoire – UserStore.java

4. Cycle de Fonctionnement

4.1 Diagramme de flux

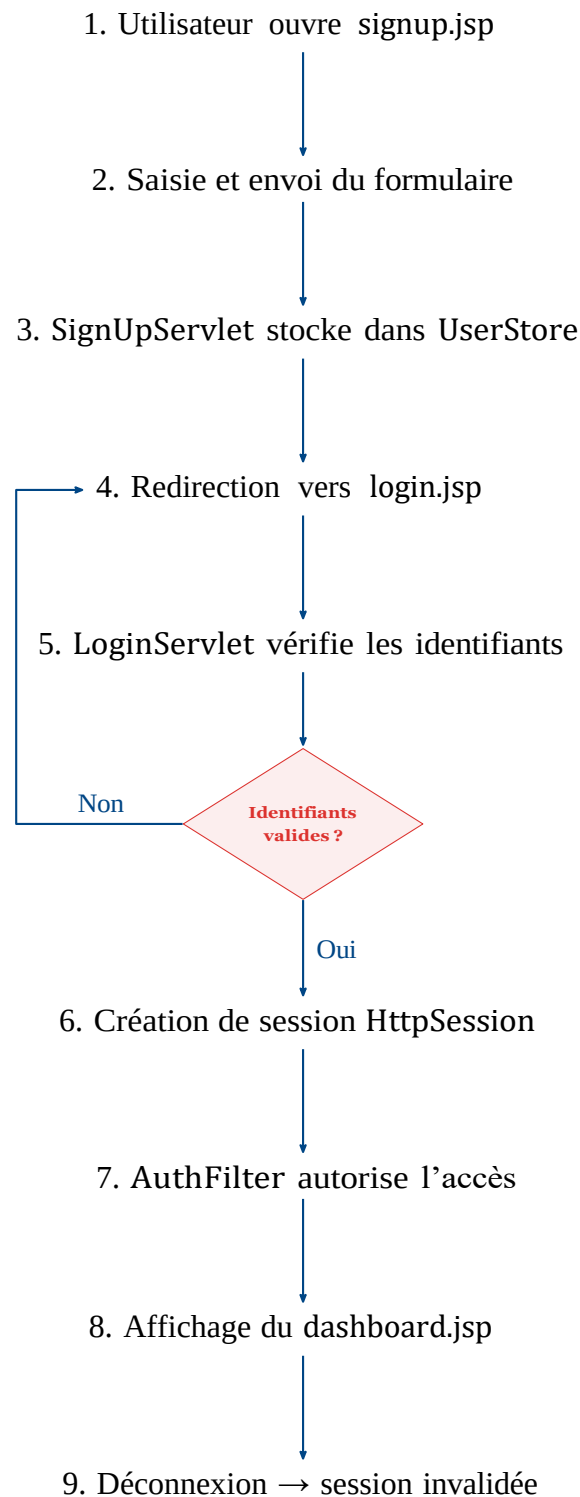


Figure 4.1. Cycle de fonctionnement de l'application d'authentification

4.2 Description étape par étape

1. L'utilisateur accède à `signup.jsp` et remplit le formulaire d'inscription.
2. Les données sont soumises à `SignUpServlet` qui les stocke dans `UserStore`.

3. L'utilisateur est redirigé vers login.jsp.
4. Il saisit ses identifiants ; LoginServlet les vérifie.
5. En cas de succès, une session HttpSession est créée.
6. AuthFilter intercepte la requête vers le dashboard et valide la session.
7. L'utilisateur accède au dashboard.jsp.
8. Il peut se déconnecter : la session est invalidée et il est redirigé vers login.jsp.

5. Difficultés & Solutions

Difficultés rencontrées

1. **Gestion des sessions** : comprendre le cycle de vie d'une HttpSession et les cas d'expiration.
2. **Sécurisation par filtre** : configurer correctement @WebFilter pour ne protéger que les routes sensibles.
3. **Organisation du code** : séparer clairement la logique métier, la présentation et la sécurité.
4. **Vérification des identifiants** : garantir la cohérence entre addUser() et authenticate().
5. **Navigation entre pages** : gérer les redirections et les *forward* sans casser l'état de la session.

Solutions apportées

1. Utilisation systématique de request.getSession(false) pour éviter les créations de session inutiles.
2. Application de @WebFilter("/dashboard/*") pour cibler uniquement les ressources protégées.
3. Création d'une classe dédiée UserStore respectant le principe de responsabilité unique.
4. Structuration MVC stricte isolant chaque couche applicative.
5. Implémentation d'un flux de navigation clair avec des redirections explicites après chaque action.

6. Résultats & Conclusion

6.1 Résultats obtenus

L'application développée lors de ce TP offre les fonctionnalités suivantes :

Fonctionnalité	Statut	Remarque
Inscription		Stockage en mémoire via UserStore
Connexion		Session créée après authentification
Déconnexion		Session invalidée, redirection vers login
Dashboard protégé		Accessible uniquement après connexion
Filtre de sécurité		AuthFilter opérationnel
Gestion des erreurs		Redirection en cas d'identifiants invalides

Table 6.1. Récapitulatif des fonctionnalités implémentées

6.2 Conclusion

Ce TP a permis d'acquérir une maîtrise concrète des mécanismes fondamentaux de la programmation web Java :

- La gestion du cycle de vie des **Servlets** et des **JSP** dans un conteneur Tomcat.
- L'utilisation des **sessions HTTP** pour maintenir l'état utilisateur.
- La mise en place d'une **sécurité par filtre** protégeant les ressources sensibles.
- L'organisation d'un projet selon le patron **MVC**.

L'application constitue une **base solide et extensible** pour le développement d'applications web sécurisées en Jakarta EE. Les prochaines améliorations envisageables incluent la persistance des données en base (JDBC/JPA), le hachage des mots de passe, et l'intégration de JSF pour les vues.